SENG 116- Object Oriented Programming

HW0: Environment Setup and Your First C# Solution

Student Name: Beren Aydemir

Student ID: 240201415

Course: SENG 116- Object Oriented Programming

Submission Date: October 18, 2026

GitHub Repository: https://github.com/berenaydemir/SENG_116_OOP.git

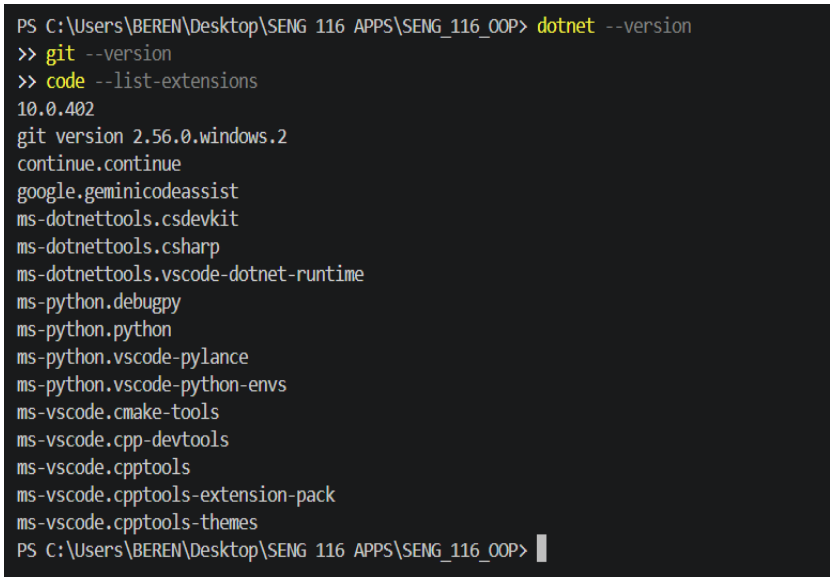
1. Objective

The objective of this assignment is to configure a complete .NET 10 development environment using Visual Studio Code and Git on Windows. It involves creating a single unified repository named SENG_116_OOP, structuring a multi-project solution containing HW0.App and HW0.Tests, implementing initial domain classes (StudentCard and NumberTools), and verifying their behavior through automated unit tests with xUnit.

2. Environment

The development and runtime environments were verified using terminal CLI commands:

- **Operating System:** Microsoft Windows 10.0.26300 (Windows 11)
- **Editor / IDE:** Visual Studio Code (C# Dev Kit extension)
- **.NET SDK:** 10.0.402
- **.NET Runtime:** .NET 10.0.13
- **Version Control System:** Git 2.56.0



3. Method

1. Initialized a single Git repository at the root directory SENG_116_OOP (git init) and created a .gitignore file using the dotnet new gitignore command to ignore unnecessary build files.
2. Created the HW0 folder, adding the HW0.slnx solution file, the HW0.App console application, and the HW0.Tests xUnit test project inside it.
3. Established a project reference from HW0.Tests to HW0.App via the dotnet add reference command so that the test project can access the classes in the main application.
4. Implemented the required class logic inside HW0.App and verified functionality against the course test suite.

4. Implementation Details

- **StudentCard.cs:** Encapsulates student data with read-only properties Name and Number. Values are assigned via the constructor, and the Summary () method returns the formatted details.
- **NumberTools.cs:** Provides static helper methods:
    - DigitSum(long number): Converts negative inputs to absolute values and iteratively sums the digits using modulo and division operations.
    - IsPrime(int number): Excludes numbers ≤1, accepts 2, rejects even numbers, and checks odd divisors up to the root of n.
    - CountPrimes(int upperBound): Excludes bounds < 2 and counts primes up to the upper bound by invoking IsPrime within a loop.

5. Tests Summary

- **Initial State:** Executing dotnet test against the skeleton implementations resulted in 4 passing and 10 failing tests. The passing cases were baseline assertions returning default placeholder values (0 and false).
- **Final State:** Following complete implementation of NumberTools.cs, all 14 xUnit tests passed with zero failures and zero warnings: Passed! - Failed: 0, Passed: 14, Skipped: 0, Total: 14

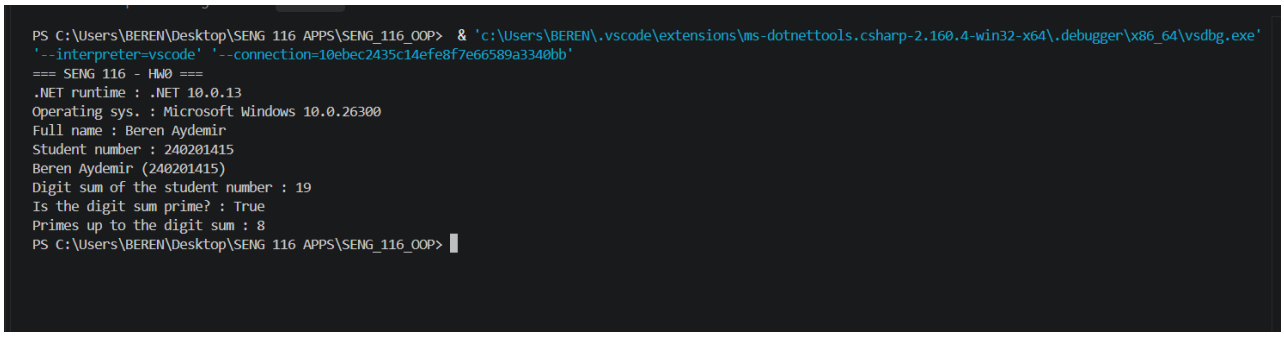
```
PS C:\Users\BEREN\Desktop\SENG 116 APPS\SENG_116_OOP> dotnet test
Geri yükleme tamamlandı (0,7sn)
  HW0.App net10.0 başarılı(1,9sn) → HW0.App\bin\Debug\net10.0\HW0.App.dll
  HW0.Tests net10.0 başarılı(0,8sn) → HW0.Tests\bin\Debug\net10.0\HW0.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.13)
[xUnit.net 00:00:01.70]   Discovering: HW0.Tests
[xUnit.net 00:00:01.75]   Discovered:  HW0.Tests
[xUnit.net 00:00:01.77]   Starting:    HW0.Tests
[xUnit.net 00:00:03.01]   Finished:    HW0.Tests
  HW0.Tests testi net10.0 başarılı (7,5s)

Test özeti: toplam: 14; başarısız: 0; başarılı: 14; atlandı: 0; süre: 7,5s
"11,2" sn'de başarılı oluşturun
PS C:\Users\BEREN\Desktop\SENG 116 APPS\SENG_116_OOP>
```

6. Experiments

Experiment 1 — Interactive Run

- **Predict:** Providing the student's name and a valid student number will instantiate StudentCard and output correct mathematical calculations.
- **Run:** Executed dotnet run --project HW0.App with inputs Beren Aydemir and 240201415.
- **Observe:** The application successfully printed:
  - Card Summary: Beren Aydemir (240201415)
  - Digit sum: 19
  - Primality of digit sum: True
  - Primes up to digit sum: 8
- **Explain:** Inputs were successfully parsed and processed through the verified static routines in NumberTools.cs.



Experiment 2 — Invalid Input

- **Predict:** Supplying alphabetical input for the student number prompt will cause parsing to fail and terminate the program.
- **Run:** Executed dotnet run --project HW0.App and entered ABC at the student number prompt.
- **Observe:** The application terminated with an unhandled exception: Unhandled exception. System.FormatException: The input string 'ABC' was not in the correct format. at Program.cs:line 12.
- **Explain:** The method long.Parse() expects a valid base-10 numerical string. Supplying non-digit characters without defensive validation or exception handling triggers a fatal runtime error.



Experiment 3 — Build Output and Git

- **Files inside HW0.App/bin/Debug/net10.0/:**
  - HW0.App.dll: The compiled Common Intermediate Language (CIL) assembly.
  - HW0.App.exe: The platform-specific executable launcher.
  - HW0.App.pdb: The program database containing debugging symbols and source mappings.
  - HW0.App.deps.json & runtimeconfig.json: Configuration files defining the dependency graph and target framework runtime parameters.
- **Git Observation:** Executing git status reports a clean working tree without listing any compiled files.
- **Explain:** The .gitignore file contains exclusion rules for bin/ and obj/ directories, preventing transient build outputs from being tracked by version control.

7. Analysis Questions

1. In the StudentCard class, which members represent state and which represent behaviour? Why is it a good design to prevent modification of name and number after creation?

- **Answer:** The properties of Name and Number represent the object's state, while the Summary () method represents its behavior. Enforcing immutability via read-only properties ensures that identity data remains constant throughout the object's lifecycle, preserving data integrity and preventing unintentional side effects across the application.

2. What are two design advantages of having CountPrimes call IsPrime rather than embedding prime-checking logic directly inside its loop?

- **Answer:**
  - *Maintainability & Reusability (DRY Principle):* Centralizing the trial-division algorithm inside IsPrime eliminates duplicate code; any optimization or bug fix applied to the primality check automatically benefits all consumers.
  - *Separation of Concerns:* CountPrimes focuses strictly on range of iteration and counting, delegating the decision-making logic of individual primality testing to a dedicated unit.

3. What exact exception occurred in Experiment 2, on which file and line did it originate, and why? What should a production-ready program do instead?

- **Answer:** The runtime threw an unhandled System.FormatException at line 12 of Program.cs. This occurred because long.Parse() requires a string composed exclusively of valid base-10 digits and cannot parse alphabetical text (ABC). A production-ready implementation should employ long.TryParse() or a defensive try-

catch block to gracefully detect invalid formats, display a user-friendly error message, and prompt re-entry without terminating execution.

4. What are the key distinctions between dotnet build, dotnet run, and dotnet test? Why are bin/ and obj/ excluded from version control, and why is deleting them safe?

- **Answer:**
  - dotnet build: Compiles source code and dependencies into executable binary assemblies (IL bytecode) without running them.
  - dotnet run: Compiles the project (if changes are detected) and immediately executes the target application.
  - dotnet test: Compiles both source and test projects, launches the configured test runner (xUnit), and outputs execution results.
  - The bin/ and obj/ directories contain transient build outputs and intermediate compiler artifacts rather than source code. They are excluded via .gitignore to avoid repository bloat and merge conflicts. Deleting them is entirely safe because they can be regenerated from source files at any time using dotnet build.

5. Why did 4 tests pass initially before writing any method bodies? Describe a specific case where a failing test helped identify an implementation issue.

- **Answer:** The initial 4 passing tests verified baseline boundary conditions where the expected result matched the skeleton placeholder return values (0 for numeric methods and false for boolean methods). During development, unit tests targeting inputs smaller than 2 failed in CountPrimes, highlighting a missing early boundary guard clause (if (upperBound < 2) return 0;) and enabling prompt correction.

8. Problems, Errors, and Solutions

- **Missing Project Reference:** When HW0.Tests was first instantiated, it could not access domain types from HW0.App.
  - *Solution:* Resolved by linking dependencies via the CLI command:

dotnet add HW0.Tests/HW0.Tests.csproj reference HW0.App/HW0.App.csproj.

- **Directory Naming Discrepancy:** The test project folder was initially created with localized non-ASCII naming.
  - *Solution:* Renamed the physical folder to HW0.Tests and updated path references inside HW0.slnx.

9. Conclusion

The development environment (.NET 10, VS Code, C# Dev Kit, and Git) was successfully configured and verified. The solution architecture was structured using modular projects adhering to object-oriented principles. Both StudentCard and NumberTools components were thoroughly verified against 14 automated xUnit test cases, achieving full test compliance and demonstrating standard software engineering workflows.

10. References and Assistance Received

- SENG 116 Course Notes and HW0 Specification Document.
- Microsoft .NET and C# Official Documentation.
- An AI coding assistant (Gemini) was consulted for CLI syntax verification, runtime exception analysis, and report drafting review.

Appendix A — Git History

```
PS C:\Users\BEREN\Desktop\SENG 116 APPS\SENG_116_OOP> git log --oneline -n 5
a53e576 (HEAD -> main, origin/main) feat: complete student card, number tools and experiments
9ad71d2 feat: setup HW0 solution and test projects
PS C:\Users\BEREN\Desktop\SENG 116 APPS\SENG_116_OOP>
```